EE5203 L05 Practice Worksheet

HAL GPIO, pulls, polling, and register evidence

Basri Erdogan

Expected time: 30-45 minutes **Policy:** Attempt every question before consulting the worked solutions. Register questions are paper questions — predict first, verify in the lab.

Part A - Reading HAL configuration

1. For each fragment, state every register field it changes (register name, bit positions, new value):

```
/* (a) PA5 */                          /* (b) PC13 */
config.Pin   = GPIO_PIN_5;             config.Pin  = GPIO_PIN_13;
config.Mode  = GPIO_MODE_OUTPUT_PP;    config.Mode = GPIO_MODE_INPUT;
config.Pull  = GPIO_NOPULL;            config.Pull = GPIO_PULLDOWN;
config.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOA, &config);         HAL_GPIO_Init(GPIOC, &config);
```

2. Write the complete `GPIO_InitTypeDef` block (with the clock enable and `HAL_GPIO_Init`) that makes **PB0** a push-pull output with no pull and low speed, and state which register fields it writes, at which bit positions.

3. Why does the generated `MX_GPIO_Init` call `HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET)` **before** `HAL_GPIO_Init`? What glitch does this ordering prevent?

4. `{0}` in `GPIO_InitTypeDef config = {0};` — what does it do, and what class of bug appears when it is omitted and `config` is reused for a second pin?

Part B - The two interfaces

5. Give the direct CMSIS equivalent of each HAL call (masks computed):

   a. `HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);`
   b. `HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);`
   c. reading B1: `HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13)` — write the `IDR` test that yields the same information.

6. Name one concrete situation where direct register access is the better choice and one where HAL is — with a one-sentence justification each. ("Better" must refer to something checkable: code size, clarity, portability, atomicity, review effort.)

Part C - Inputs and pulls

7. For each wiring, state what the pin reads when the switch is open and when it is closed:

   a. resistor to 3.3 V, switch to ground (B1's wiring);
   b. resistor to ground, switch to 3.3 V;
   c. no resistor, switch to ground.

   Which case is a bug, and what would the program observe?

8. The board's B1 has external R30 to 3.3 V. A student nevertheless configures `GPIO_PULLUP`. Explain what is now physically on the pin, and whether pressed/released readings change.

9. Fill the truth table for the Lab 4 Checkpoint B loop:

B1	`ReadPin` returns	branch taken	LD2
released			
pressed			

Then state the one-token change that swaps the behavior (LD2 blinks while B1 is held and is off after release).

Part D - Counting presses

10. This loop is meant to count presses but counts hundreds per press:

```
if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) == GPIO_PIN_RESET) {
    press_count = press_count + 1;
}
```

    a. Why does it over-count even with a perfect, bounce-free button?
    b. Rewrite the loop body with `previous` and `current`, as in Lab 4 Checkpoint C, so that only the start of a press is counted.
    c. With that fix, the count still jumps by 2-3 per press. What causes it, and where does the `HAL_Delay(20)` go?

11. In Checkpoint C, `press_count` and `previous` are declared in `USER CODE BEGIN 2`, before the loop. What breaks if they are declared inside the body of `while (1)` instead?

12. `press_count` starts at 0 and B1 is pressed five times. Using `press_count % 4` and the Checkpoint C behaviors (0 off · 1 on · 2 blink every 200 ms · 3 blink every 50 ms), which behavior is active after each press?

Part E - Register evidence

13. After `MX_GPIO_Init()` with the Lab 4 settings (PA5 output, PC13 input, no pull), predict the exact values of these fields:

Field	Predicted
`GPIOA->MODER` bits 11:10	
`GPIOA->OTYPER` bit 5	
`GPIOC->MODER` bits 27:26	
`GPIOC->PUPDR` bits 27:26	

    Check your predictions in XPERIPHERALS in the lab (Registers Tree at home).

14. While you hold B1 down, which single SFR bit changes, in which register, and does `GPIOC->ODR` change too? Why or why not?

15. Your claim: "HAL wrote the same MODER bits that my Lab 3 `led_init` wrote by hand." Describe the two-project comparison (breakpoints, registers, values) that proves it.

Exit self-assessment

Mark one:

- ☐ Ready for Lab 4 without additional review
- ☐ Need to review HAL-name-to-register mapping
- ☐ Need to review pulls and the B1 circuit
- ☐ Need to review press counting and contact bounce
- ☐ Need to review the register evidence workflow